B.TECH COMPUTER SCIENCE ENGINEERING

Backend Development — Node.js, Express.js & MongoDB

CASE STUDY 55

FitMirror

Virtual Fitting Room

Project Report & API Documentation

Submitted by	Ayush Mishra
Roll number	150096725046
Program	B.Tech Computer Science Engineering (CSE)
Batch	2025–29
Institution	ITM Skills University
Course	Backend Development — Node.js, Express.js & MongoDB

Contents

1. Abstract . . . 3
2. Introduction . . . 3
3. Requirements and Implementation Status . . . 4
4. Technology Stack . . . 5
5. System Architecture . . . 5
6. Backend Project Structure . . . 6
7. Database Design . . . 6
8. REST API Reference . . . 8
9. Authentication, Authorization and Validation . . . 9
10. Measurements and Recommendation Logic . . . 9
11. Analytics and Real-Time Notifications . . . 10
12. End-to-End Usage Flow . . . 10
13. API Documentation . . . 10
14. API Testing Evidence . . . 11
15. Testing Summary . . . 15
16. Design Decisions and Limitations . . . 15
17. Future Scope . . . 15
18. Conclusion . . . 16
19. References . . . 16

1. Abstract

Online fashion retail suffers from a persistent problem: shoppers cannot tell how a garment will fit before they buy it. FitMirror is a Virtual Fitting Room backend, built for Case Study 55 of the Backend Development course, that addresses this by combining stored body measurements with a transparent, rule-based size recommendation engine.

The system is a RESTful API written in **Node.js and Express.js** with **MongoDB** (via Mongoose) for persistence. Authentication uses **JWT** and **Firebase Email/Password** sign-in, with role-based authorization separating shoppers from retailers. Users maintain measurements, receive size and garment recommendations, save and share looks, and upload photos. Retailers manage garments and view conversion analytics, and **Socket.io** supports real-time notifications.

The API is documented through both a Postman collection and an interactive Swagger UI, and the key flows were verified against the running backend. This report describes the requirements, design, implementation, testing evidence, scope decisions and future work.

23	6	2	6
REST endpoints	MongoDB collections	user roles	verified API calls

2. Introduction

2.1 Background

Size uncertainty is one of the main reasons for returns in online fashion. A virtual fitting room reduces that uncertainty by letting a shopper store their measurements once and see which sizes and garments suit them. On the business side, retailers need to know whether virtual try-ons actually turn into purchases.

2.2 Problem statement

Develop the backend API for “FitMirror” so that users can upload photos or use a webcam, try virtual clothes with size recommendations, save outfits and share looks; retailers can upload garments and track conversions; and images are stored reliably.

2.3 Objectives

- Build a modular Node.js + Express.js REST API with separate routes, controllers, middleware, services and models.
- Model users, garments, measurements, saved looks, notifications and analytics events in MongoDB using Mongoose.
- Secure the API with JWT, Firebase authentication, role-based authorization and ownership checks.
- Provide measurement-driven size recommendations and garment recommendations that are easy to explain and reproduce.

- Support saved and shareable looks, retailer conversion analytics and real-time notifications.
- Document and test the API using Postman and Swagger UI.

3. Requirements and Implementation Status

The table below maps every requirement of the supplied case study to its implementation status in the final project.

Requirement	Implementation / status
Node.js + Express.js REST API	Implemented with modular routes, controllers, middleware, models and services.
MongoDB + Mongoose	Implemented for users, garments, measurements, saved looks, notifications and analytics events.
JWT authentication	Implemented. Login returns a FitMirror JWT used for protected requests.
Firebase Auth	Implemented with Firebase Email/Password sign-in and a backend ID-token exchange endpoint.
Garment CRUD	Implemented. Public reads; retailer-only create/update/delete with ownership checks on update and delete.
Measurements + size recommendation	Implemented using stored measurements and rule-based size computation.
Saved looks + sharing	Implemented with saved looks and share-token based public viewing.
Recommendations	Implemented as deterministic, rule-based recommendations (no ML dependency).
Analytics	Try-on and purchase-click events, plus retailer conversion analytics.
Notifications / Socket.io	Notification API and Socket.io service for real-time delivery.
File upload middleware	Implemented with Multer and local backend storage.
Firebase Storage	Not used. Local Multer storage is used instead to avoid Firebase Blaze billing for Cloud Storage.
Optional advanced CV / AI features	Not implemented; the brief marks them optional and the project is scoped to the required features.
API documentation	Delivered as a Postman collection and an interactive Swagger UI.

3.1 Deliverables checklist

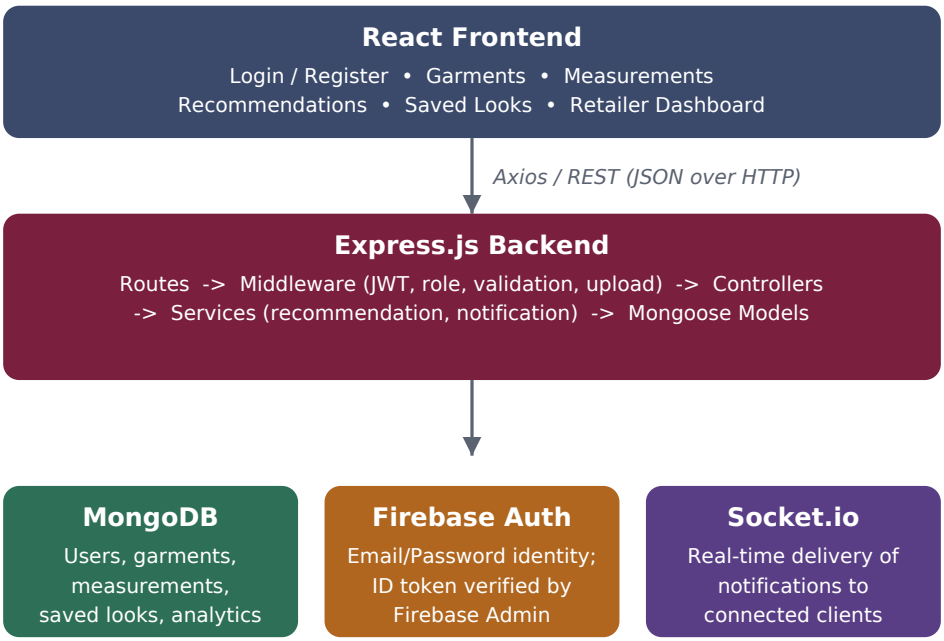
Deliverable (from the brief)	Status
REST API with all CRUD operations	**Completed**
API documentation (Swagger / Postman collection)	**Completed**
GitHub repository with README and setup instructions	Repository link: _____
Bonus: deployment on Render / Heroku	Not attempted (bonus item)

4. Technology Stack

Layer	Technology	Role in FitMirror
Runtime	Node.js	Server-side JavaScript runtime for the API.
Web framework	Express.js	Routing, middleware pipeline and REST endpoints.
Database	MongoDB + Mongoose	Document storage with schema validation through the Mongoose ODM.
Authentication	JWT, Firebase Authentication	Stateless API sessions plus Firebase Email/Password identity.
Real time	Socket.io	Pushes notifications to connected clients.
File uploads	Multer	Handles multipart image uploads, stored locally.
Validation	Request validators	Reject malformed input before it reaches controllers.
Documentation	Swagger UI, Postman	Interactive API reference and a reusable request collection.
Frontend	React + Axios	Separate user and retailer flows consuming the REST API.

5. System Architecture

FitMirror follows a client–server architecture. The React frontend talks to the Express backend over REST. MongoDB holds all persistent data, Firebase provides the authentication identity, and Socket.io delivers real-time notifications.



Each request passes through a fixed pipeline: **routes** map a URL to a handler; **middleware** authenticates the caller, checks the role, validates input and processes uploads; **controllers** orchestrate the response; **services** hold reusable business logic such as the recommendation and notification rules; and **models** persist data. Keeping these concerns separate makes each part easy to test and change.

6. Backend Project Structure

The backend is organised by responsibility instead of placing all logic in a single file.

```
backend/
|-- src/
|   |-- config/        # database, Firebase, Socket.io, environment
|   |-- controllers/   # request / response logic
|   |-- middleware/    # auth, role, validation, upload, errors
|   |-- models/        # Mongoose models
|   |-- routes/        # REST API route definitions
|   |-- services/      # recommendation and notification logic
|   |-- validators/    # request validation rules
|   |-- utils/         # API response / error helpers
|   |-- seed.js        # sample data
|   `-- server.js      # server entry point
`-- uploads/           # locally stored uploaded images
```

The frontend provides pages for authentication, garments, measurements, recommendations, saved looks, shared looks and the retailer dashboard, together with reusable API, authentication and UI components.

7. Database Design

MongoDB stores six kinds of documents, each represented by a Mongoose model.

Collection	Purpose	Key fields
Users	Accounts for shoppers and retailers.	name, email, role (user / retailer), photoUrl, createdAt
Garments	Catalogue items uploaded by retailers.	title, description, category, price, availableSizes, imageUrl, purchaseUrl, retailer
Measurements	One body-measurement profile per user.	height, weight, chest, waist, hips, shoulder, preferredFit
Saved looks	Outfits a user saves and can share.	owner, selected garments, share token
Notifications	Messages sent by retailers to users.	recipient, message, read status
Analytics events	Try-on and purchase-click tracking.	event type, garment, user (optional), timestamp

8. REST API Reference

All endpoints are served under `/api`. “Public” endpoints need no token; protected endpoints expect `Authorization: Bearer <token>`.

Module	Method	Endpoint	Access
Auth	POST	`/api/auth/register`	Public
	POST	`/api/auth/login`	Public
	POST	`/api/auth/firebase`	Public (Firebase ID token)
	GET	`/api/auth/me`	Authenticated
Garments	GET	`/api/garments`	Public
	GET	`/api/garments/:id`	Public
	POST	`/api/garments`	Retailer
	PUT	`/api/garments/:id`	Retailer (owner)
	DELETE	`/api/garments/:id`	Retailer (owner)
Measurements	GET	`/api/measurements`	Authenticated
	PUT	`/api/measurements`	Authenticated
Recommendations	GET	`/api/recommendations`	Authenticated
	GET	`/api/recommendations/size`	Authenticated
Saved looks	POST	`/api/saved-looks`	Authenticated
	GET	`/api/saved-looks`	Authenticated
	GET	`/api/saved-looks/:id`	Authenticated; share-token view is public
	DELETE	`/api/saved-looks/:id`	Authenticated
Analytics	POST	`/api/analytics/track`	Authentication optional
	GET	`/api/analytics/conversions`	Retailer
Notifications	POST	`/api/notifications/send`	Retailer
	GET	`/api/notifications`	Authenticated
	PUT	`/api/notifications/:id/read`	Authenticated
User	POST	`/api/user/upload`	Authenticated

Garment listing supports pagination (page, `limit`) and filters for category, size, free-text search and retailer, as shown in the Postman evidence in Section 13.

9. Authentication, Authorization and Validation

- **JWT authentication.** A successful login returns a FitMirror JWT. Protected endpoints read it from the Authorization header using the Bearer scheme.
- **Firebase authentication.** The frontend signs users in with Firebase Email/Password. The Firebase ID token is sent to `POST /api/auth/firebase`, where Firebase Admin verifies it; the backend then finds or creates the matching application user and issues the same FitMirror JWT.
- **Role-based authorization.** Retailer-only actions pass through role middleware. Public registration does not let a client choose the retailer role, which prevents self-promotion.
- **Ownership checks.** A retailer can update or delete only garments they own.
- **Validation.** Request validators are wired into the routes. Invalid ObjectIds are returned as client errors instead of crashing the server.
- **File uploads.** Multer processes image uploads and stores them in the backend uploads directory.

> **Defence in depth**
> A protected write passes through several independent checks: a valid JWT, the required role, ownership of the resource, and request validation. Failing any one of them stops the request before the database is touched.

10. Measurements and Recommendation Logic

Users store height, weight, chest, waist, hips, shoulder and a preferred fit. The recommendation endpoints turn these values into a recommended size and a short fit explanation. The approach is deliberately rule-based rather than a machine-learning or computer-vision model: it is deterministic, simple to explain, and easy to reproduce in a viva.

Input	Processing	Output
Chest, waist, preferred fit	Rule-based size computation	Recommended size and fit advice
User measurements + garment availability	Match garments against the recommended size, with optional category / search filters	List of recommended garments

For example, with a chest of 92 cm, a waist of 78 cm and a regular fit preference, the size endpoint returns size S together with the explanation “Based on your chest (92cm) and waist (78cm) with a regular fit preference.” (see Figure 6).

> **Why rule-based?**
> A rule-based engine needs no training data, always gives the same answer for the same measurements, and can be explained line by line during evaluation — which suits the scope of this case study.

11. Analytics and Real-Time Notifications

Analytics. Try-on and purchase-click events are recorded through POST `/api/analytics/track` (authentication optional, so anonymous browsing can still be measured). Retailers read aggregated results from GET `/api/analytics/conversions`, which answers the business question in the brief: how many try-ons convert into purchases.

Notifications. Retailers send notifications through the API; they are stored in MongoDB so users can list them and mark them as read, and the Socket.io service delivers them in real time to connected clients.

12. End-to-End Usage Flow

The flow below follows the example user flow from the case study and shows how the modules fit together.

Step	Actor	Request	Outcome
1	User	`POST /api/auth/login`	Receives a JWT.
2	User	`POST /api/user/upload`	Uploads a photo, stored by Multer.
3	User	`PUT /api/measurements`	Measurements saved.
4	User	`GET /api/recommendations`	Gets a recommended size and matching garments.
5	User	`POST /api/saved-looks`	Saves an outfit and can share it by token.
6	Retailer	`GET /api/analytics/conversions`	Sees how try-ons convert to purchases.

13. API Documentation

Postman. A complete FitMirror collection is organised into Authentication, Garments, Measurements, Recommendations, Saved Looks, Analytics, Notifications and User folders. It defines reusable variables such as `baseUrl`, `token`, `garmentId`, `savedLookId` and `notificationId`.

Swagger / OpenAPI. The running backend serves an interactive Swagger UI at `http://localhost:5001/api-docs`, listing every endpoint with its description, authentication requirement (lock icon) and response schemas. Together these satisfy the

Swagger/Postman documentation requirement of the case study.

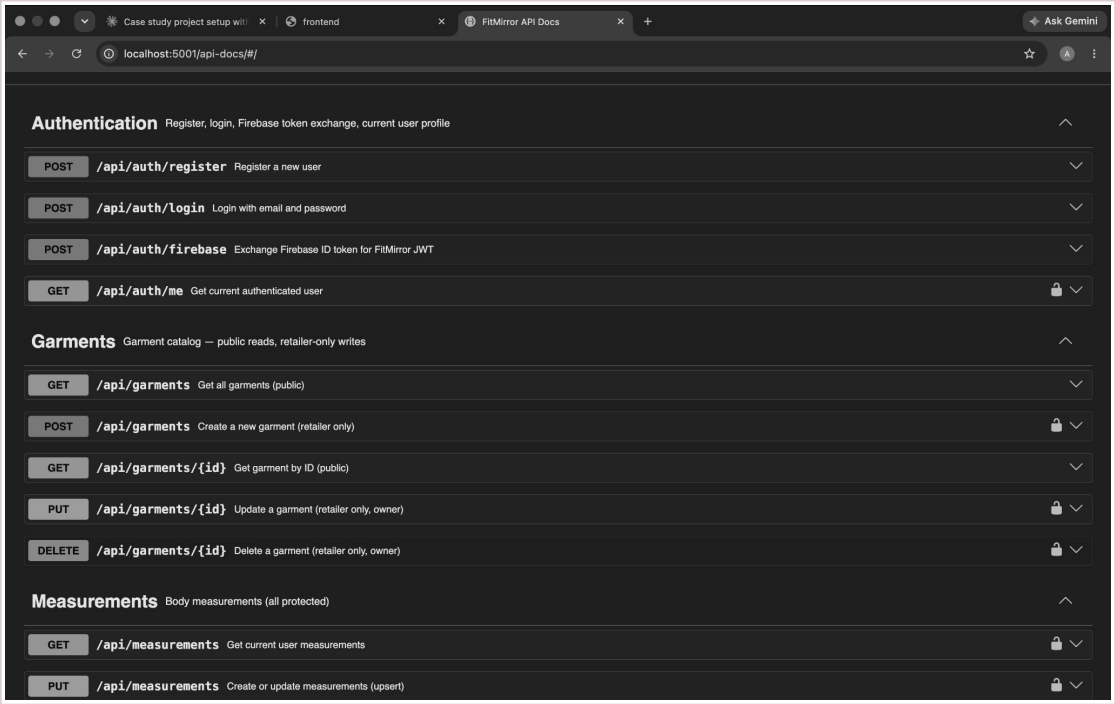


Figure 1. Swagger UI — FitMirror API documentation at /api-docs.

The documentation groups endpoints into Authentication, Garments and Measurements (continuing with the remaining modules below the fold). Padlock icons mark protected routes.

14. API Testing Evidence

The following screenshots were captured while testing the local backend (`http://localhost:5001/api`) through Postman. They show representative successful calls rather than every endpoint. Credentials in the request body are masked.

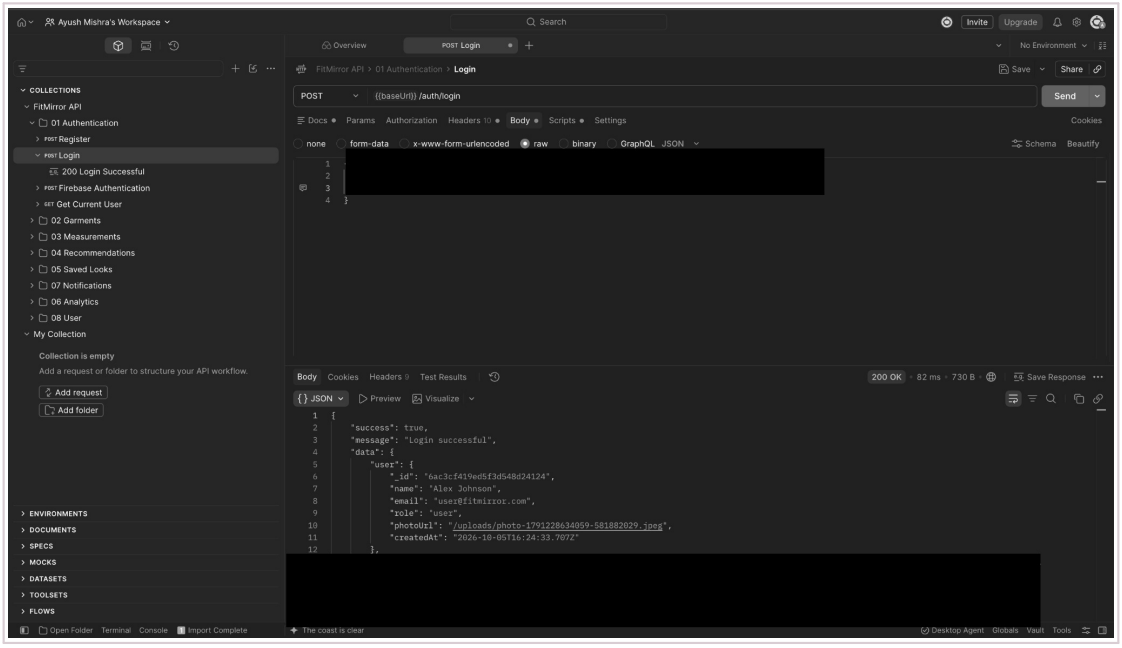


Figure 2. Postman — POST /auth/login returns 200 OK.

The login request returns *“Login successful”* with the user profile (id, name, email, role, photo URL) and the issued JWT.

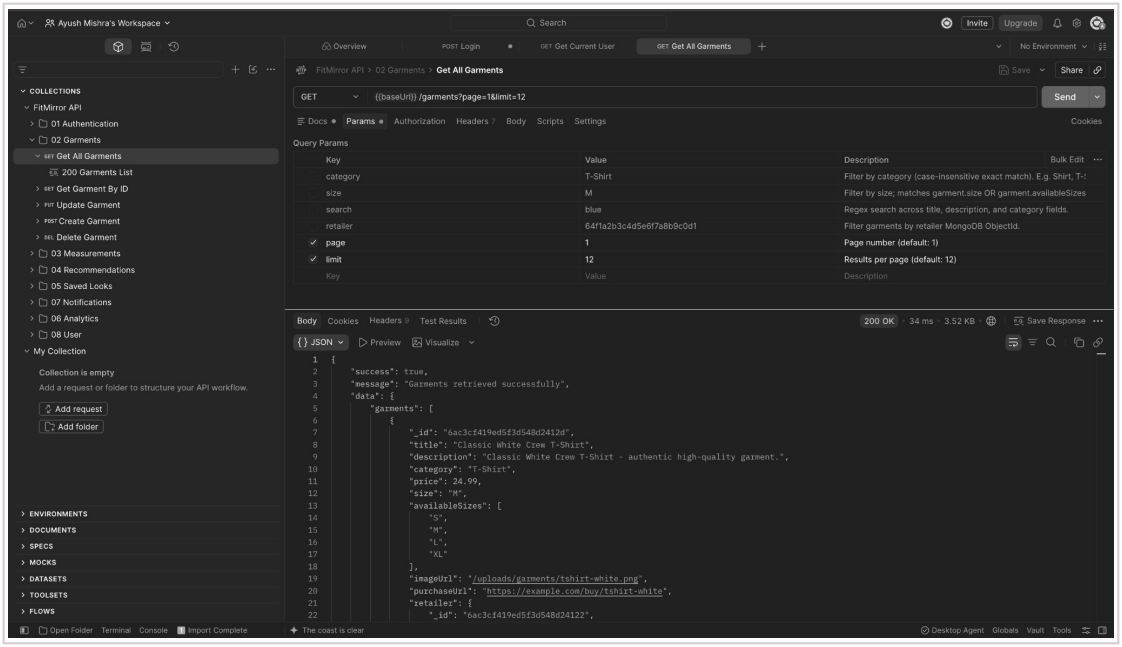


Figure 3. Postman — GET /garments?page=1&limit=12 returns 200 OK.

The catalogue endpoint is public and paginated. The Params tab lists the supported filters (category, size, search, retailer, page, limit); each garment includes title, category, price, available sizes, image URL and purchase URL.

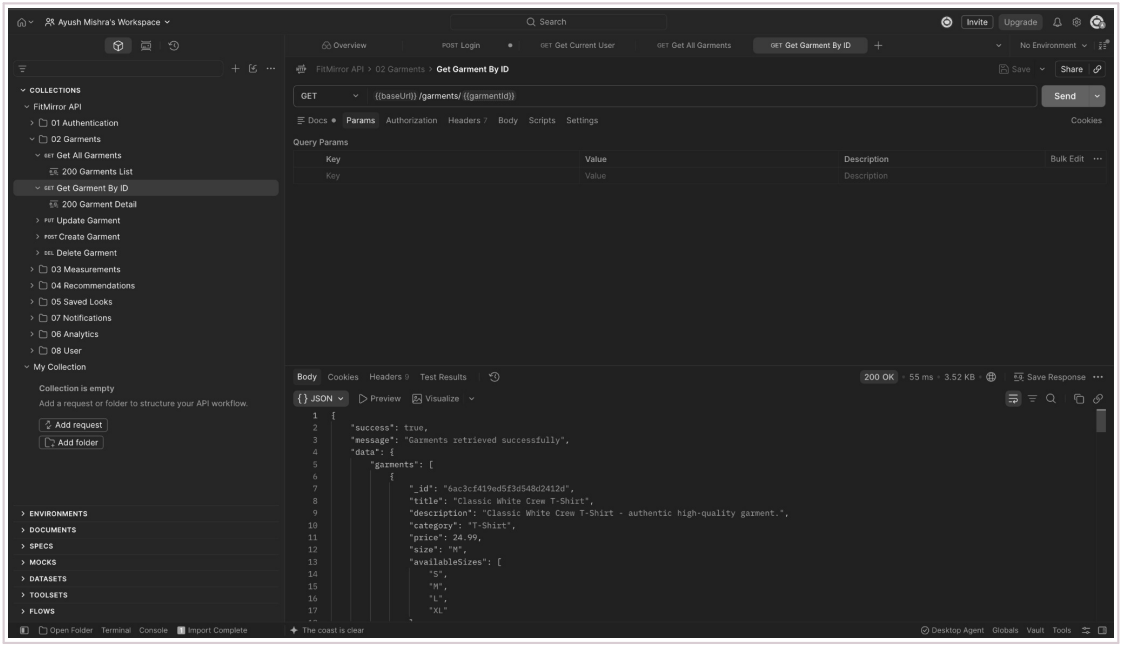


Figure 4. Postman — GET /garments/{{garmentId}} returns 200 OK.

Fetching a single garment by its identifier returns the garment details, using the collection variable `garmentId`.

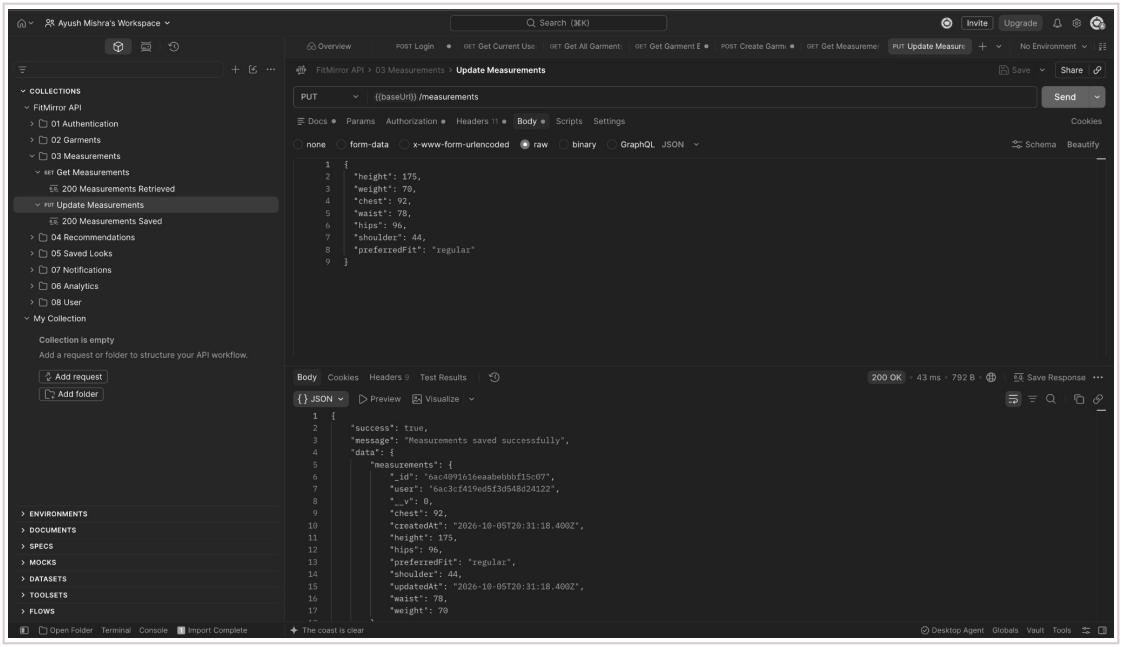


Figure 5. Postman — PUT /measurements returns 200 OK.

The upsert endpoint saves height, chest, waist, hips and preferred fit and responds with *“Measurements saved successfully”*.

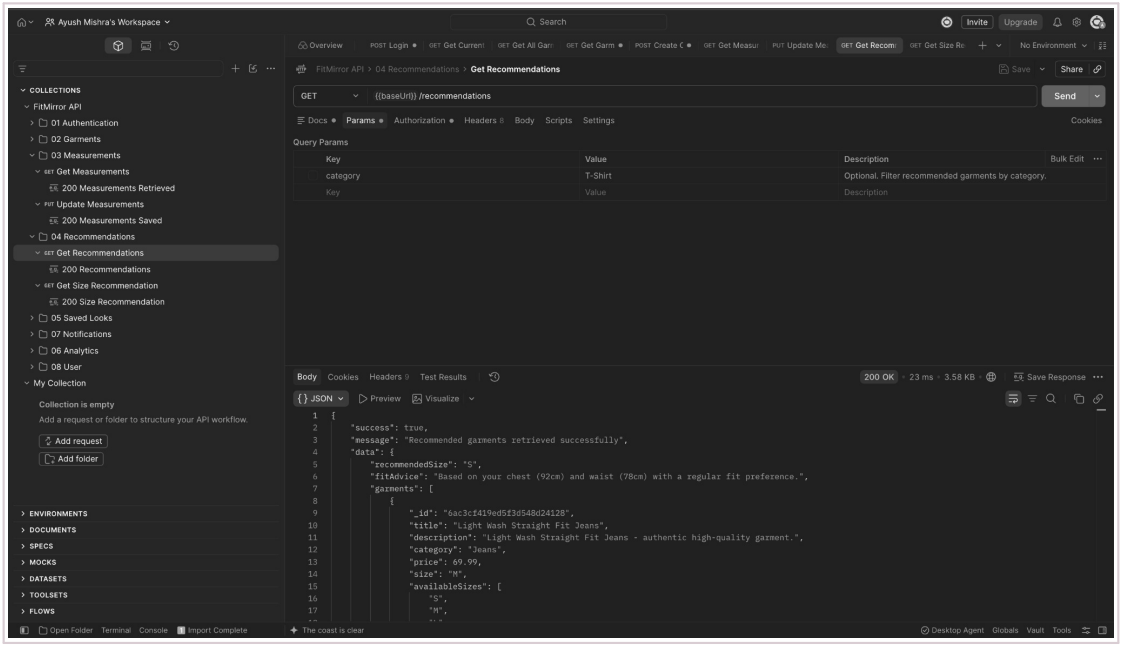


Figure 6. Postman — GET /recommendations returns 200 OK.

The response contains the recommended size (S), the fit advice derived from chest 92 cm and waist 78 cm, and a list of matching garments. An optional `category` parameter filters the list.

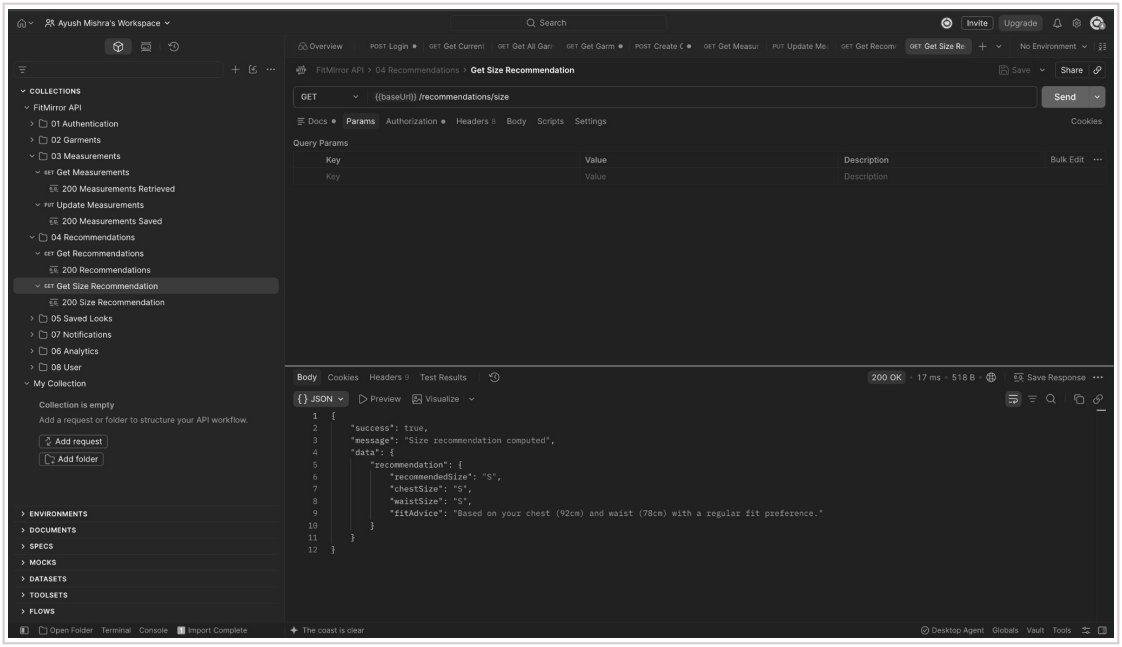


Figure 7. Postman — GET /recommendations/size returns 200 OK.

The lightweight size endpoint returns the recommended size together with the chest size and waist size it was derived from, plus the fit advice text.

15. Testing Summary

Tested operation	Result
Login	200 OK; success message, user data and JWT returned.
Get all garments	200 OK; paginated garment list returned.
Get garment by ID	200 OK; garment details returned.
Update measurements	200 OK; measurements saved.
Get recommendations	200 OK; recommended size, fit advice and garments returned.
Get size recommendation	200 OK; computed size and fit advice returned.
Swagger UI	Loaded at /api-docs and displayed the documented API groups.

These results demonstrate working request/response behaviour for authentication, catalogue retrieval, measurement persistence, recommendation logic and interactive documentation.

16. Design Decisions and Limitations

- **Local storage instead of Firebase Storage.** Images are stored with Multer on the backend to avoid requiring Firebase Blaze billing for Cloud Storage.
- **Rule-based recommendations.** Size advice is computed from measurements by explicit rules, not by a model, so every result can be explained.
- **Optional features excluded.** Body measurement from a photo, ML style recommendations and computer-vision try-on are listed as optional in the brief and were intentionally left out.
- **No deployment.** Deployment to Render or Heroku is a bonus item and was not required for the local documentation and testing phase.
- **Scope.** The project prioritises clear architecture, working REST APIs, authentication, validation, CRUD operations and explainable logic.

17. Future Scope

- Integrate Firebase Cloud Storage or another production object store when billing and deployment allow.
- Add computer-vision based body measurement from uploaded photographs.
- Build a genuine virtual try-on pipeline using a suitable vision model.
- Add ML-based style recommendations using behaviour and garment metadata.
- Deploy the backend and database to production infrastructure with secure environment variables.
- Add automated API tests and CI checks for regression testing.

18. Conclusion

FitMirror implements the core backend requirements of the Virtual Fitting Room case study using Node.js, Express.js, MongoDB/Mongoose, JWT, Firebase Email/Password authentication, Multer uploads and Socket.io notifications. It offers modular REST APIs, role-based retailer features, measurement-driven size recommendations, saved and shareable looks, analytics, notifications, and documentation through both Postman and Swagger UI.

By deliberately avoiding optional AI/CV features, the project stays understandable, reproducible and easy to demonstrate, while leaving a clear path for the future enhancements listed above.

19. References

- Express.js documentation — https://expressjs.com
- Mongoose documentation — https://mongoosejs.com/docs
- MongoDB manual — https://www.mongodb.com/docs/manual
- Firebase Authentication — https://firebase.google.com/docs/auth
- Socket.io documentation — https://socket.io/docs
- JSON Web Tokens — https://jwt.io/introduction
- Multer (file uploads) — https://github.com/expressjs/multer
- Swagger / OpenAPI — https://swagger.io/docs
- Postman learning center — https://learning.postman.com
- Case Study 55: Backend Development — Virtual Fitting Room, B.Tech CSE, ITM Skills University.